GUÍA DE INSTALACIÓN / 01

APP CENSO Reconocimiento facial local

Manual de configuración y puesta en marcha
Windows + Python + OpenCV + Supabase + HTML

Preparado para Carlos Mauricio Delgado · 25 de octubre de 2026.

Qué documenta esta guía

El procedimiento que funcionó en las pruebas: instalación del motor en una PC, carga de modelos, procesamiento de fotografías guardadas en Supabase y búsqueda desde la página web con inicio de sesión.

Alcance: guía para configurar el proyecto existente. Los archivos HTML, JavaScript y Python se obtienen de tu copia de GitHub o del respaldo del proyecto. Este PDF no reemplaza esos archivos ni incluye todas sus líneas de código.

Estado comprobado

Componente	Resultado confirmado
Acceso y registro	Inicio de sesión, autorización, personas y fotos funcionando.
Motor local	Detección de un rostro y descriptor de 128 valores.
Búsqueda web	HTML → Python → Supabase → candidato con similitud.
Pendiente	Umbral, descriptor automático al registrar e historial de consultas.

Capacidad prevista: hasta 5.000 personas y un operador, una consulta a la vez. La capacidad completa y los tiempos no se han validado con 5.000 registros.

Regla de lectura: ejecutá cada comando por separado. No copiés el texto PS C:\...> que muestra la terminal.

1. Preparar la PC y el proyecto

Equipo utilizado en la prueba

Componente	Configuración
Procesador	Intel Core i3-9100F, 4 núcleos
Memoria	16 GB de RAM
Disco	SSD SATA
Gráfica	NVIDIA GeForce GT 710; las pruebas se prepararon para CPU.
Software observado	Python 3.12.10 · OpenCV 5.0.0

La PC necesita Internet para consultar Supabase. Debe permanecer encendida cuando se use el reconocimiento. Bloquear Windows o apagar la pantalla permite seguir ejecutando el servicio; suspender, hibernar o apagar la PC lo interrumpe.

1.1. Elegir la carpeta definitiva

Abrí el proyecto existente desde GitHub en esta ubicación. Los archivos de la página quedan en la raíz y el motor en una subcarpeta:

```
C:\Proyectos\GitHub\APP_CENSO\
    index.html
    style.css
    app.js
    config.js
    .gitignore
    APP_CENSO_MOTOR\
        main.py
        motor_facial.py
        procesar_fotos.py
        buscar_local.py
        api_busqueda.py
        .env
        .venv\
        modelos\
            yunet.onnx
            sface.onnx
```

GitHub guarda el código; no mantiene el servicio Python encendido. En otra PC se recrea el entorno, se recuperan los modelos y se configura un nuevo .env.

2. Descargar e instalar Python

2.1. Descargar el instalador usado

Abrí la página oficial:
https://www.python.org/downloads/release/python-31210/

En Files, descargá **Windows installer (64-bit)**. La versión 3.12.10 es la que funcionó en esta instalación; no se presenta como la versión más reciente.

2.2. Instalar

Ejecutá el instalador. Antes de pulsar Install Now, marcá **Add python.exe to PATH**. Terminá la instalación y cerrá la terminal que estuviera abierta.

2.3. Verificar desde una terminal nueva

```
python --version

python -m pip --version
```

Resultado comprobado: Python 3.12.10 y pip instalado. El aviso de una versión nueva de pip no significa que la instalación falló.

Si Windows abre Microsoft Store

El mensaje “Python was not found” significa que ese comando no está encontrando la instalación. Revisá la instalación y PATH, y abrí una terminal nueva. El acceso directo de Microsoft Store no demuestra que Python esté instalado.

2.4. Abrir el proyecto

En VS Code: Archivo → Abrir carpeta → C:\Proyectos\GitHub\APP_CENSO. Usá la terminal PowerShell integrada para los siguientes pasos.

3. Crear el entorno del motor

En PowerShell, ejecutá una línea a la vez. Si la carpeta ya existe, entrá en ella; no hace falta crearla de nuevo.

```
cd C:\Proyectos\GitHub\APP_CENSO

mkdir APP_CENSO_MOTOR

cd APP_CENSO_MOTOR

python -m venv .venv
```

3.1. Instalar las dependencias

```
.\.venv\Scripts\python.exe -m pip install "fastapi[standard]"

.\.venv\Scripts\python.exe -m pip install opencv-python-headless

.\.venv\Scripts\python.exe -m pip install supabase python-dotenv
```

Estos comandos instalan las versiones disponibles al ejecutarlos. Para reproducir las mismas versiones en otra PC, conservá un requirements.txt generado desde el entorno que funciona (ver página 12).

3.2. Comprobar el entorno y las dependencias

```
.\.venv\Scripts\python.exe --version

.\.venv\Scripts\python.exe -m pip --version

$prueba = @'
import fastapi, uvicorn, cv2
from supabase import create_client
from dotenv import load_dotenv
print('Dependencias correctas')
'@
.\.venv\Scripts\python.exe -c $prueba
```

La ruta de pip debe contener APP_CENSO_MOTOR\.venv. No hace falta activar el entorno: usar su python.exe explícitamente evita ejecutar otro Python por error.

Si movés la carpeta: conservá el código y recreá .venv en la nueva ubicación. No copies un entorno virtual entre equipos.

4. Descargar los modelos

Dentro de APP_CENSO_MOTOR, creá la carpeta modelos:

```
New-Item -ItemType Directory -Force modelos
```

YuNet detecta el rostro. SFace genera su descriptor. Para OpenCV 5 usamos la variante YuNet 2026may.

4.1. Descargar YuNet

```
Invoke-WebRequest -Uri (
  "https://github.com/opencv/opencv_zoo/raw/refs/heads/main/" +
  "models/face_detection_yunet/" +
  "face_detection_yunet_2026may.onnx"
) -OutFile ".\modelos\yunet.onnx"
```

4.2. Descargar SFace

```
Invoke-WebRequest -Uri (
  "https://github.com/opencv/opencv_zoo/raw/refs/heads/main/" +
  "models/face_recognition_sface/" +
  "face_recognition_sface_2021dec.onnx"
) -OutFile ".\modelos\sface.onnx"
```

4.3. Verificar los archivos

```
Get-ChildItem .\modelos | Select-Object Name, Length
```

Archivo	Tamaño observado en bytes
yunet.onnx	229738
sface.onnx	38696353

Estos tamaños son una comprobación inicial, no una verificación criptográfica. Si descargaste una página HTML o un puntero Git LFS en vez del modelo, la carga fallará. Conservá los modelos exactos que ya funcionaron para futuras reinstalaciones.

5. Preparar Supabase

En una reinstalación que use el mismo proyecto de Supabase, **no ejecutes de nuevo el script inicial de tablas**. La base y las fotografías permanecen en la nube aunque cambies de PC.

5.1. Si el proyecto es completamente nuevo

Creá el proyecto y ejecutá el script inicial APP_CENSO_Tablas.sql que se preparó para la app. Crea usuarios_autorizados, personas, fotografias, consultas y coincidencias, sus relaciones y permisos, y el bucket privado fotos-personas.

Después ejecutá el SQL de descriptores_faciales que usamos en la sesión. Esa tabla contiene fotografia_id, modelo, descriptor real[] de 128 valores y creado_en; tiene RLS activo y acceso directo reservado a service_role.

5.2. Crear y autorizar el usuario

En Authentication → Users → Add user → Create new user, creá la cuenta del operador. Confirmá el usuario según las opciones del panel. Después ejecutá en SQL Editor, reemplazando el correo:

```
INSERT INTO public.usuarios_autorizados (id, nombre, rol)
SELECT id, 'Carlos', 'administrador'
FROM auth.users
WHERE lower(email) = lower('TU_CORREO_AQUI')
RETURNING id, nombre, rol, activo;
```

Debe devolver una fila con activo = true. No vuelvas a insertar la misma cuenta si ya está autorizada. Las contraseñas pertenecen a Supabase Auth y no a la tabla usuarios_autorizados.

5.3. Comprobar las fotos

El bucket se llama **fotos-personas** y es privado. La tabla fotografias guarda la ruta del objeto y la persona relacionada. Las fotos de prueba ya registradas se recuperan desde ahí.

Antes del uso real, deshabilitá el registro público de cuentas. Tener una cuenta de Auth no basta: también debe estar activa en usuarios_autorizados.

6. Configurar claves y GitHub

6.1. Archivo .env del motor

Creá .env dentro de APP_CENSO_MOTOR, junto a main.py. Conservá estos nombres exactos. En una instalación distinta, cambiá la URL por la del proyecto correspondiente.

```
SUPABASE_URL=https://rjvncqmbxlvwsykgrtxx.supabase.co
SUPABASE_PUBLISHABLE_KEY=TU_CLAVE_PUBLICA
SUPABASE_SERVER_KEY=TU_CLAVE_PRIVADA_DE_SERVIDOR
```

La clave pública se obtiene en API Keys. SUPABASE_SERVER_KEY usa la clave Secret o la antigua service_role. Pegala solo en este archivo local; no se incluye ninguna clave privada en este PDF.

6.2. Archivo .gitignore de la raíz

En C:\Proyectos\GitHub\APP_CENSO\.gitignore, agregá estas líneas sin borrar otras reglas:

```
.env
.venv/
__pycache__/
```

Si un archivo ya se subió a GitHub, .gitignore no lo retira del historial. Si alguna vez se publicó una clave privada, hay que revocarla o rotarla, además de retirar el archivo del seguimiento.

6.3. Diferencia con config.js

config.js pertenece a la página y contiene solamente la URL y la clave publicable, con las propiedades que usa app.js:

```
window.CENSO_CONFIG = Object.freeze({
  url: "https://rjvncqmbxlvwsykgrtxx.supabase.co",
  key: "TU_CLAVE_PUBLICA"
});
```

La clave privada nunca va en config.js, app.js ni index.html. El navegador envía la sesión del operador al motor; Python valida esa sesión antes de buscar.

7. Recuperar los archivos probados

Usá la versión del proyecto que pasó las pruebas de esta sesión. No mezcles los primeros scripts de prueba con las versiones que usan el motor compartido.

Archivo	Responsabilidad
motor_facial.py	Carga YuNet y SFace; límite de 10 MB; escala máxima de 640 píxeles; exige un rostro; genera y normaliza 128 valores.
procesar_fotos.py	Lee .env, conecta con clave privada y genera o actualiza descriptores de las fotos registradas.
buscar_local.py	Permite consultar una foto desde la terminal y listar candidatos. Usa generar_descriptor y MODELO.
api_busqueda.py	Define POST /buscar; valida sesión y autorización; devuelve hasta tres personas candidatas.
main.py	Crea FastAPI, configura CORS e incluye el router de búsqueda.
app.js	Inicia sesión, registra personas/fotos y envía la foto con Authorization al motor.

7.1. Comprobar el motor compartido

```
$prueba = @'
from motor_facial import generar_descriptor
print('Motor compartido preparado')
'@
.\.venv\Scripts\python.exe -c $prueba
```

7.2. Procesar las fotografías existentes

```
.\.venv\Scripts\python.exe procesar_fotos.py
```

Resultado confirmado con una foto:
Actualizadas: 1 | Por revisar: 0 | Errores: 0

La versión final del script actualiza descriptores existentes. Las fotos con cero o varios rostros quedan para revisión. No borra personas ni fotografías.

Limitación actual: el HTML todavía no genera automáticamente el descriptor al guardar una persona. Por ahora, después de registrar nuevas fotos, ejecutá procesar_fotos.py antes de buscarlas.

8. Configurar y arrancar la API

En main.py debe existir una sola creación de app. Las rutas /salud y /detectar que ya tiene el archivo pueden conservarse. El bloque central que funcionó es:

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from api_busqueda import router

app = FastAPI(title="APP CENSO - Prueba local")

app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://127.0.0.1:3000",
        "http://localhost:3000",
        "http://127.0.0.1:5500",
        "http://localhost:5500"
    ],
    allow_methods=["POST"],
    allow_headers=["Authorization", "Content-Type"]
)

app.include_router(router)
```

No agregués una segunda línea app = FastAPI(...) debajo: reemplazaría la aplicación que tiene CORS. El origen incluye protocolo, nombre/IP y puerto; no incluye la ruta index.html ni parámetros.

8.1. Iniciar desde la carpeta correcta

```
cd C:\Proyectos\GitHub\APP_CENSO\APP_CENSO_MOTOR

.\.venv\Scripts\python.exe -m uvicorn main:app --host 127.0.0.1 --port 8000
```

Esperá “Uvicorn running on http://127.0.0.1:8000”. Dejá la terminal abierta. Ctrl+C detiene el servicio. Después de modificar Python, detenelo y volvé a iniciarlo.

Abrí http://127.0.0.1:8000/docs. Debe aparecer POST /buscar con candado. /detectar es una prueba de detección; no confirma identidad ni reemplaza /buscar.

9. Abrir la página y probar

9.1. Iniciar la vista web

Abrí index.html con VS Code Live Preview o Live Server. La dirección que funcionó fue:

```
http://127.0.0.1:3000/APP_CENSO/index.html
?vscode-livepreview=true
```

La URL anterior se muestra en dos líneas por espacio; en el navegador es una sola dirección. Si tu servidor usa 5500 también está autorizado en la configuración anterior.

9.2. Comprobar el botón de búsqueda

La versión probada de app.js usa photoBlob y la sesión de Supabase. La llamada principal tiene esta estructura:

```
const { data, error } = await db.auth.getSession();
if (error || !data.session) {
    throw new Error("Volvé a iniciar sesión.");
}
const formulario = new FormData();
formulario.append("foto", photoBlob, "consulta.jpg");
const respuesta = await fetch("http://127.0.0.1:8000/buscar", {
    method: "POST",
    headers: {
        Authorization: `Bearer ${data.session.access_token}`
    },
    body: formulario
});
```

Este fragmento explica la conexión; no reemplaza el controlador completo del botón que maneja errores, espera y presentación de resultados. No fijes Content-Type manualmente al enviar FormData.

9.3. Hacer una búsqueda completa

1. Iniciá sesión con el operador autorizado.
2. Tomá o subí una fotografía con un rostro.
3. Pulsá Buscar coincidencias · Prueba.
4. Revisá el candidato y la similitud.

Resultado web confirmado: Prueba-001 con similitud 0.9035. Sin umbral, el sistema presenta candidatos ordenados; incluso una persona diferente puede aparecer si es el único registro disponible. No equivale a una identidad confirmada.

10. Resolver los errores que vimos

Mensaje / síntoma	Qué hacer
Python was not found	Instalar Python con PATH, cerrar y abrir la terminal.
Could not import module main	Entrar a APP_CENSO_MOTOR y verificar dir main.py antes de iniciar Uvicorn.
No se reconoce .\.venv\Scripts\python.exe	El entorno no está en esa carpeta. Crear .venv ahí e instalar dependencias. El texto (.venv) en el prompt puede ser de otra ubicación.
ImportError: detector desde procesar_fotos	En buscar_local.py usar from procesar_fotos import db y from motor_facial import generar_descriptor, MODELO. Retirar el procesamiento antiguo.
Targets are not supported...	En las pruebas aparecieron como WARN y el procesamiento terminó. No confundir con un traceback que detenga el programa.
Se necesita exactamente un rostro	Mostrar cantidad detectada. Usar una foto frontal, clara y con una persona. La escala 640 permitió detectar la imagen cercana que fallaba a 1200.
OPTIONS /buscar 405	Revisar que CORS esté en la app que realmente ejecuta Uvicorn. Guardar y reiniciar; no crear otra app después del middleware.
CORS rechaza el origen	Autorizar el puerto real del HTML. En esta sesión era 3000, no 5500.
curl: (7) Could not connect	El servicio no está escuchando. Iniciarlo y dejarlo activo; ejecutar curl en otra terminal.
401 / 403 al buscar	Volver a ingresar; comprobar usuario activo en usuarios_autorizados y la validación del token.
503 al buscar	Revisar conexión con Supabase, clave del servidor, permisos y terminal de Python.

No agregues OPTIONS a una ruta para ocultar el problema: CORSMiddleware atiende el preflight cuando está cargado y la solicitud incluye Origin y Access-Control-Request-Method.

11. Rutina diaria y respaldo

Cada vez que vayas a usar la app

1. Encendé la PC y verificá Internet.
2. Abrí la terminal en APP_CENSO_MOTOR.
3. Iniciá Uvicorn con el comando de la página 9.
4. Abrí la página desde Live Preview/Live Server.
5. Iniciá sesión y probá una consulta.
6. Tras registrar fotos nuevas, ejecutá procesar_fotos.py en otra terminal mientras la integración automática siga pendiente.

Prueba de CORS desde otra terminal

```
curl.exe -i -X OPTIONS "http://127.0.0.1:8000/buscar" `
  -H "Origin: http://127.0.0.1:3000" `
  -H "Access-Control-Request-Method: POST" `
  -H "Access-Control-Request-Headers: authorization"
```

Los acentos graves al final de cada línea son continuación de PowerShell. Debe responder 200 OK y access-control-allow-origin: http://127.0.0.1:3000.

Guardar las versiones que funcionan

```
.\.venv\Scripts\python.exe -m pip freeze > requirements.txt
```

En otra PC, después de crear .venv y recuperar requirements.txt:

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

Conservá código, requirements.txt y los modelos. Recreá .env localmente. No subas la clave privada. La PC nueva no queda lista solo por abrir GitHub; necesita completar la configuración y arrancar el servicio.

Límites y trabajo pendiente

127.0.0.1 apunta al equipo que abre la página: esta configuración está probada solo en la misma PC. Faltan acceso remoto HTTPS, arranque como servicio, historial, automatización de descriptores, consentimiento y umbral validado. La prueba actual es de comparación facial; no demuestra detección de vida ni resistencia a fotos de otra pantalla.

12. Lista final de comprobación

Comprobación	Resultado esperado
Python	python --version → Python 3.12.10 en el equipo probado.
Entorno	pip apunta a APP_CENSO_MOTOR\.venv.
Dependencias	Importaciones sin errores.
Modelos	yunet.onnx y sface.onnx presentes y cargables.
Supabase	URL y claves correctas; .env solo en el motor.
Autorización	Operador existente en Auth y activo en usuarios_autorizados.
Datos	Fotos privadas, metadatos y descriptores relacionados.
Procesamiento	Un rostro; descriptor normalizado de 128 valores.
API	Uvicorn activo en 127.0.0.1:8000; /buscar visible.
CORS	Preflight 200 para el origen real de la página.
HTML	Foto + sesión enviadas al motor; resultados visibles.
Respaldo	Código, modelos y requirements conservados; .env excluido.

Referencias oficiales

Python 3.12.10
https://www.python.org/downloads/release/python-31210/

OpenCV Zoo - modelos y licencias
https://github.com/opencv/opencv_zoo

FastAPI - CORS
https://fastapi.tiangolo.com/tutorial/cors/

FastAPI - archivos
https://fastapi.tiangolo.com/tutorial/request-files/

Supabase - cliente Python
https://supabase.com/docs/reference/python/introduction

Documento basado en los pasos y resultados confirmados durante la configuración. No certifica precisión de identificación ni rendimiento con la base completa. Las claves privadas y las fotografías de participantes no se incluyen.